

Turan User Guide

Turan is a Python-based web security scanner and hardening assistant.

This guide is a living document. It describes the current CLI behavior, the command set, the important flags, and what to expect from each workflow.

For version history, see docs/changelog.md.

This guide and the changelog should be updated together when Turan's user-facing behavior changes.

What Turan Does

Turan helps you:

- scan a target URL
- crawl in-scope pages
- discover a local app target on a VPS when no URL is supplied
- inspect local environment and server layout
- generate safe fix artifacts
- apply one real local fix lane with backup and validation
- save reports, baselines, audit history, and comparison summaries

Turan is intentionally defensive:

- it warns before risky commands
- it never prints secret values
- it prefers read-only behavior unless you explicitly ask for fixes
- it keeps a clear trail in the terminal and in saved outputs

Command Map

Command	What it does	Typical use
---	---	---
'scan'	Scans one target page or a discovered local target	Quick vulnerability check
'crawl'	Crawls in-scope links across multiple pages	Broader site sweep
'report'	Re-renders or previews a saved report	Review an existing JSON, Markdown, or HTML
'audit'	Shows the append-only audit trail	Review scans and fix events
'baseline'	Saves a scan snapshot for later comparison	Track a known-good state
'compare'	Compares two saved scans and crawl coverage	See what changed
'doctor'	Checks the local machine and app environment	Health check without a URL
'server-check'	Discovers the server layout and scans the local app target	VPS/server discovery
'fix --local'	Applies the first real local edit lane	Backup, edit, validate, rollback if needed

| 'demo-site' | Runs the local demo site | Test target for development |

Quick Start

Scan a target:

```
.\env\Scripts\python.exe -m app.main scan https://example.com
```

Discover a VPS target without typing a URL:

```
.\env\Scripts\python.exe -m app.main scan
```

Crawl more than one page:

```
.\env\Scripts\python.exe -m app.main crawl https://example.com --max-pages 20 --max-depth 2
```

Generate an HTML report with a default filename under 'outputs/':

```
.\env\Scripts\python.exe -m app.main crawl https://example.com --html-output
```

Run the server discovery workflow:

```
.\env\Scripts\python.exe -m app.main server-check --yes
```

Apply the first real local fix lane:

```
.\env\Scripts\python.exe -m app.main fix --local --yes
```

Safety and Permission Prompt

Risky commands ask for confirmation by default:

- 'scan'
- 'crawl'
- 'server-check'
- 'fix --local'

The prompt is a reminder that you should only test systems you own or have explicit permission to test.

Use '--yes' to skip the prompt in trusted automation.

Read-only commands stay quiet:

- 'doctor'
- 'report'
- 'audit'
- 'compare'
- 'baseline'

Target Resolution

'scan' and 'crawl' accept a target URL directly.

If you omit the URL, Turan resolves a target in this order:

1. explicit URL on the command line
2. '--env-file'
3. the apps own '.env'
4. OS environment variables
5. server discovery on a VPS

The common environment variables are:

- 'APP_URL'
- 'TARGET_URL'
- 'BASE_URL'

When Turan discovers a local app target, it prints a short 'Discovery:' line and then a fuller application context block.

Output Files and Paths

Turan writes user-facing artifacts into 'outputs/'.

Supported output flags:

- '--json-output'
- '--markdown-output'
- '--html-output'
- '--output'
- '--audit-log'
- '--log-file'

You can use them in three ways:

- pass a full path
- pass a relative path
- pass the flag without a path and let Turan create a timestamped filename under 'outputs/'

Examples:

```
.\env\Scripts\python.exe -m app.main scan https://example.com --html-output
.\env\Scripts\python.exe -m app.main crawl https://example.com --json-output outputs\crawl.js
.\env\Scripts\python.exe -m app.main baseline https://example.com --output
```

If you use a Windows-rooted path like '\outputs\crawl.html', Turan treats it as project-relative and prints the resolved path.

'scan'

'scan' checks a single target endpoint.

Syntax:

```
.\env\Scripts\python.exe -m app.main scan [URL]
```

Important flags:

- '--env-file PATH' reads a specific '.env'
- '--timeout SECONDS' changes the request timeout
- '--policy PATH' loads a policy file
- '--yes' skips the permission prompt
- '--preview-fixes' shows proposed fixes only
- '--interactive' lets you choose generate artifacts or local fixes
- '--generate-fixes' generates safe local remediation artifacts
- '--apply-fixes' is a legacy alias for '--generate-fixes'
- '--login-url', '--auth-method', '--username', '--password', '--password-env', '--cookie', '--session-file', '--storage-state', and browser auth flags enable authenticated scanning
- '--json-output', '--markdown-output', '--html-output' save reports

What to expect:

- one target page

- headers, cookies, TLS summary, exposed files, WAF signals
- suggested fix plans when report-only findings exist
- optional saved reports in JSON, Markdown, and HTML

Example:

```
.\venv\Scripts\python.exe -m app.main scan https://example.com --json-output --markdown-output
```

‘crawl’

‘crawl’ starts from one page and follows in-scope links across multiple pages.

Syntax:

```
.\venv\Scripts\python.exe -m app.main crawl [URL]
```

Important flags:

- ‘--max-pages N’
- ‘--max-depth N’
- ‘--include REGEX’
- ‘--exclude REGEX’
- ‘--same-host-only’ and ‘--allow-offsite’
- ‘--seed-robots’ and ‘--seed-sitemap’
- ‘--env-file PATH’
- ‘--yes’
- authentication and browser-auth flags from ‘scan’
- report output flags

What to expect:

- a crawl summary with pages visited, unique URLs, and seed sources
- findings grouped by page in the terminal and saved reports
- repeated findings collapsed into an ‘Affected URLs’ list
- optional seeded discovery from ‘robots.txt’ and ‘sitemap.xml’

Example:

```
.\venv\Scripts\python.exe -m app.main crawl https://example.com --max-pages 20 --max-depth 2 -
```

Authenticated Scanning and Crawling

Turan supports generic authenticated flows for protected content.

Supported auth modes:

- HTTP JSON login
- HTTP form login
- raw cookie reuse
- saved session files
- browser storage-state reuse
- browser-assisted login for JS-heavy flows

Common flags:

- '--login-url'
- '--auth-method json|form|browser'
- '--username'
- '--password'
- '--password-env'
- '--user-field'
- '--pass-field'
- '--cookie'
- '--session-file'
- '--save-session'
- '--storage-state'
- '--save-storage-state'
- '--browser-username-selector'
- '--browser-password-selector'
- '--browser-submit-selector'
- '--browser-headless' and '--browser-headed'
- '--auth-check-url'

Recommended pattern:

```
.\venv\Scripts\python.exe -m app.main crawl https://example.com --login-url /auth/login --auth
```

What to expect:

- Turan logs in before crawling or scanning protected pages
- it confirms that login worked by checking a protected URL when you provide '--auth-check-url'
- it reuses the same session for the scan or crawl

'report'

'report' re-renders or previews a saved scan report from disk.

Syntax:

```
.\venv\Scripts\python.exe -m app.main report <REPORT_FILE>
```

Accepted inputs:

- '.json'
- '.md'
- '.html'
- '.htm'

Optional output flags:

- '--json-output'
- '--markdown-output'
- '--html-output'

What to expect:

- JSON reports are rendered back into the normal terminal report view
- Markdown and HTML files can be copied to new destinations
- bare output flags create files under 'outputs/'

'audit'

'audit' shows the append-only audit history.

Important flags:

- '--audit-log' and legacy alias '--log-file'
- '--last N'
- '--event NAME'
- '--target TEXT'
- '--json-output'

What to expect:

- filtered audit events in the terminal

- optional JSON export of the filtered audit list
- audit entries for scans, baselines, and fix actions

Example:

```
.\venv\Scripts\python.exe -m app.main audit --last 25 --json-output
```

‘baseline‘

‘baseline‘ saves a scan snapshot for later comparison.

Important flags:

- ‘--label NAME‘
- ‘--output‘
- ‘--audit-log‘
- ‘--timeout‘
- ‘--policy‘

What to expect:

- a baseline JSON snapshot
- a companion ‘.meta.json‘ metadata file
- discovery details saved alongside the snapshot

Example:

```
.\venv\Scripts\python.exe -m app.main baseline https://example.com --label vps-west --output
```

‘compare‘

‘compare‘ compares two saved scan files.

Important flags:

- ‘--markdown-output‘
- ‘--html-output‘

What to expect:

- fixed findings
- new findings

- unchanged findings
- crawl coverage deltas when the inputs come from 'crawl'

Example:

```
.\venv\Scripts\python.exe -m app.main compare old.json new.json --html-output
```

'doctor'

'doctor' checks the local machine and app environment.

What it checks:

- OS info
- Python version
- '.env' presence
- output folder writability
- app config paths
- common Nginx config paths
- localhost ports
- safe secret-status signals without printing secret values

Example:

```
.\venv\Scripts\python.exe -m app.main doctor --env-file C:\path\to\autoentrytrack\.env
```

'server-check'

'server-check' is the server-facing discovery flow.

What it does:

- checks Nginx and systemd hints
- discovers the app target
- resolves the apps own '.env' when possible
- scans the discovered local target

Important flags:

- '--env-file'
- '--nginx-config'
- '--timeout'

- '--yes'

Example:

```
.\venv\Scripts\python.exe -m app.main server-check --yes
```

'fix --local'

'fix --local' is the first real live-edit lane.

What it does:

1. discovers a supported local file
2. backs it up
3. applies the change
4. validates it
5. rolls back on failure

What to expect:

- the target file is backed up before any edit
- the terminal shows the backup path and validation command
- the chosen local fix must be supported by the current lane
- unsupported findings stay in generate-artifact mode

Example:

```
.\venv\Scripts\python.exe -m app.main fix --local --yes
```

'--generate-fixes' and the legacy '--apply-fixes'

'--generate-fixes' creates local remediation artifacts and notes.

'--apply-fixes' still works as a backwards-compatible alias, but '--generate-fixes' is the current name.

What to expect:

- a backup of the generated artifact, if one already exists
- a file under 'outputs/generated/'
- a matching note and audit entry

'demo-site'

'demo-site' runs a local test target for development.

Example:

```
.\env\Scripts\python.exe -m app.main demo-site --port 8000
```

'env' Variables

These are the main environment variables Turan reads:

Variable	Used by	Purpose
'APP_URL'	'scan'	Primary fallback target
'TARGET_URL'	'scan'	Secondary fallback target
'BASE_URL'	'scan'	Final fallback target
'DEBUG'	'doctor', 'server-check'	Warns about noisy debug mode
'SECRET_KEY'	'doctor', 'server-check'	Presence/weakness check only
'SERVER_NAME'	'doctor', 'server-check'	Reported as present or missing
'DATABASE_URL'	'doctor', 'server-check'	Reported as present or missing
'SMTP_PASSWORD'	'doctor', 'server-check'	Reported as present or missing

Saved Outputs

Current output folders:

- 'outputs/' for user-facing artifacts
- 'outputs/generated/' for generated fix artifacts
- 'outputs/backups/' for backup files
- 'outputs/remediation/' for remediation notes

Useful output examples:

```
.\env\Scripts\python.exe -m app.main scan https://example.com --html-output
.\env\Scripts\python.exe -m app.main crawl https://example.com --markdown-output outputs\crawl
.\env\Scripts\python.exe -m app.main audit --json-output
```

What the Output Means

Typical scan output includes:

- target summary
- finding counts
- severity counts
- top categories
- WAF or CDN signals
- TLS summary when available
- a findings table
- proposed fixes for report-only issues

Typical crawl output adds:

- pages visited
- unique URLs
- seed sources
- per-page findings
- grouped repeated findings with affected URL lists

Typical doctor or server-check output adds:

- root path
- OS
- Python version
- environment file discovery
- Nginx/systemd hints
- localhost port checks

Troubleshooting

If a bare output flag creates a file and you cannot find it:

- Turan writes under 'outputs/' in the current project directory
- if you pass '\outputs\file.html', Turan treats it as project-relative and tells you the resolved path
- if you pass a full absolute path, Turan respects it

If 'crawl' only reaches login or redirect pages:

- add auth flags
- add '--auth-check-url'
- confirm that the login worked before you trust the crawl results

If 'server-check' finds a local app target but still points to the repo '.env':

- make sure the discovered 'EnvironmentFile' or working directory is correct
- pass '--env-file' if you want to override discovery

Maintaining This Guide

This guide is meant to be updated as Turan grows.

When you add a new command or flag:

1. update the CLI help in 'app/main.py'
2. update this guide
3. update the README quick-start sections if needed
4. regenerate the PDF